

Chapitre II

L'environnement Unix/Linux

Plan du Chapitre

- ✓ Introduction aux systèmes « *Unix/Linux* »
- ✓ Le système de fichiers
- ✓ Sécurité du système de fichiers
- ✓ La programmation shell
- ✓ Les filtres et les expressions régulières
- ✓ Awk et Sed
- ✓ Gestion des processus

1. Introduction aux systèmes « UNIX/Linux »

Historique des systèmes UNIX(1)

- ✓ 1969: naissance d'Unix dans les laboratoires BELL
- ✓ 1973: portage du Noyau en C (Thompson et Ritchie) $\Rightarrow$ portage sur plusieurs architectures:
 - ☞ IRIX pour SGI
 - ☞ SOLARIS pour SUN
 - ☞ AIX pour IBM
 - ☞ HPUX
 - ☞ OSF pour DEC etc.

Historique des systèmes UNIX(2)

✓ Problèmes:

☞ Plusieurs versions propriétaires: Solaris (Sun), AIX (IBM), HPUNIX (HP), etc.

☞ Portabilité des applications d'une architecture à une autre

☞ Besoin de standardisation

☞ Plusieurs tentatives avec les projets

- POSIX => spec. des systèmes ouverts
- X/OPEN
- SVID (System V Interface Definition)

☞ Actuellement: 2 normalisations à vocations commerciales et industrielles

- IX/OSF (Unix d'IBM), par le groupe OSF
- Unix System V, groupe ARCHER d'AT&T

Émergence des logiciels libres

- ✓ 1984: Projet **GNU**: création d'applications libres (*open source*)
- ✓ La **GPL** (General Public Licence): licence permettant de protéger le caractère « *open source* » des logiciels libres
- ✓ La GPL assure et protège 4 libertés :
 - ☞ Utilisation
 - ☞ Duplication (pas de licence limitée en nombre d'installation)
 - ☞ Accès au code source
 - ☞ Modification et redistribution du code source
 - BSD (Berkeley): premier Unix libre
- ✓ Un logiciel libre n'est pas forcément gratuit !
 - ☞ Exemples de logiciels libres :
 - ☞ **OpenOffice** : suite bureautique
 - ☞ **VLC** : lecteur multimédia

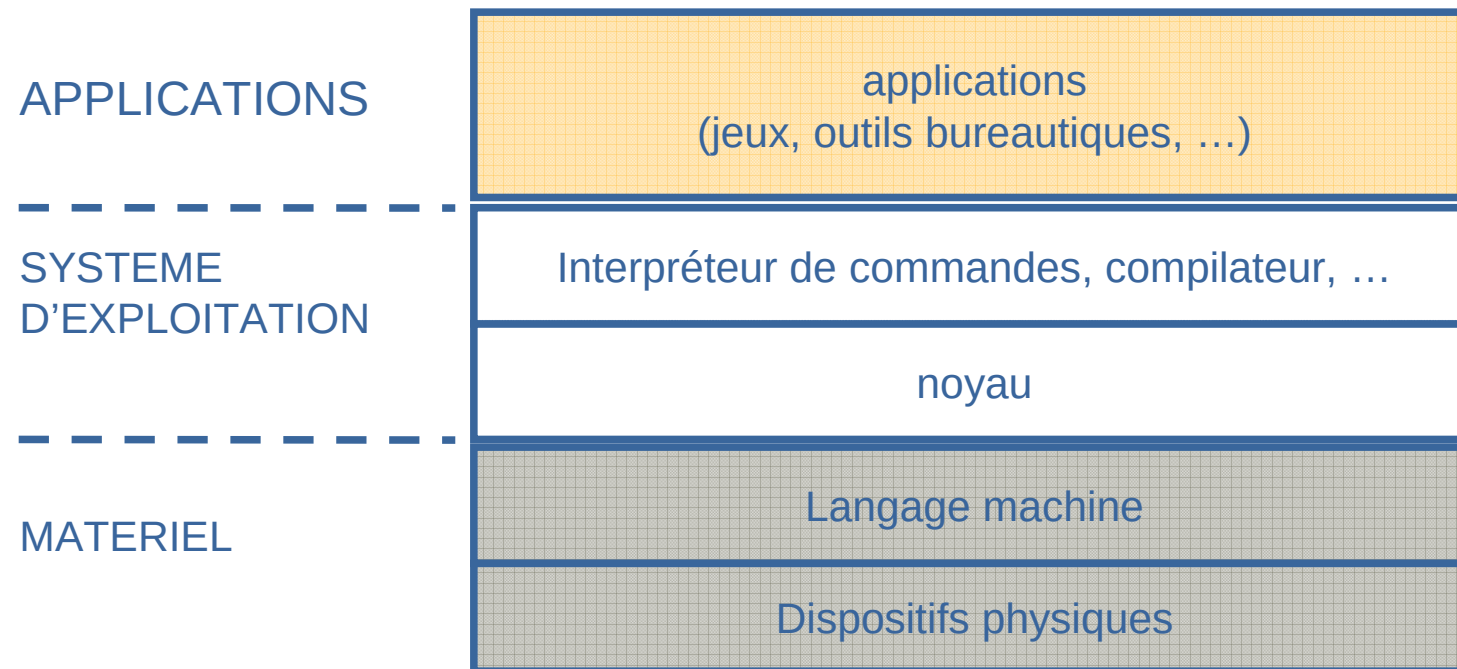
Les distributions Linux

- ✓ 1991: Linus Torvalds crée le « *Kernel* » : le noyau des systèmes Linux actuels

Kernel+ applications GNU => LINUX

- ✓ Kernel : implémentation mise à jour des API Unix (System V) pour supporter une large gamme de matériels, de pilotes et de protocoles réseaux.
- ✓ Actuellement plusieurs **distributions** existent
 - ☞ Même noyau
 - ☞ Environnement utilisateur et ensembles de logiciels utilitaires variables
- ✓ Distributions libres mais commerciales :
 - ☞ RHEL (RedHat Enterprise Edition), SuSE Linux Enterprise
- ✓ Distributions libres et gratuites : Fedora, OpenSUSE, Ubuntu, Mandriva, CentOS, Gentoo, Debian, Knoppix, etc ...

Architecture (1)



Architecture (2)

- ✓ Un système Unix est composé de:
 - ☞ Un noyau: assure la gestion de la mémoire et des entrées/sorties de bas niveau et l'ordonnancement des tâches
 - ☞ Un ensemble d'outils de base:
 - ☞ Différents interpréteurs de commande appelés *shells*;
 - ☞ Des commandes de manipulation de fichiers;
 - ☞ Des commandes de gestion des activités du système (les *processus*);
 - ☞ Des commandes permettant la communication entre utilisateurs ou entre systèmes;
 - ☞ Des éditeurs de textes;
 - ☞ Des outils de traitement de textes et de traitement d'images;
 - ☞ Des compilateurs;
 - ☞ Des lecteurs multimédias (audio, vidéo);
 - ☞ Des outils de développement;
 - ☞ Etc.

Caractéristiques (1)

- ✓ Système ouvert
- ✓ Système sécurisé
- ✓ Multitâches
- ✓ Multi-utilisateurs
- ✓ Multi-sessions
- ✓ Tout est fichier même le matériel
- ✓ Système de fichiers hiérarchisé
- ✓ Système hiérarchisé de processus
- ✓ Plusieurs interpréteurs de commandes (SHELL)

Caractéristiques (2)

✓ Les utilisateurs

- ☞ 3 types : administrateur, non privilégiés et utilisateurs système.
- ☞ La plupart (admin et non privilégiés) doivent être authentifiés par un login et un mot de passe
- ☞ Chaque utilisateur est caractérisé, par le système, par un **UID** unique
- ☞ Chaque utilisateur a un répertoire de travail par défaut (répertoire *home*)
- ☞ Rassemblés en groupes
- ☞ Un unique administrateur ou super user: login= **root** et **mot de passe choisi au moment de l'installation**
- ☞ **root** peut créer des utilisateurs non privilégiés

Caractéristiques (3)

- ✓ Possibilités d'ouvrir plusieurs sessions simultanées
- ✓ Session graphique : **Bureau**
 - ☞ Un bureau est un environnement de travail complet muni de thèmes personnalisés en plus d'un ensemble de logiciels dédiés.
 - ☞ KDE, GNOME
- ✓ Sessions textuelles : **Consoles Virtuelles**
 - ☞ Exécution de commandes sous le Shell
 - ☞ Accessibles à travers les raccourcis : CTRL+ALT+F(2-->6)
- ✓ Sessions distantes si service réseau disponible (Telnet, SSH)

Caractéristiques (4) -- Le Shell

- ✓ Interpréteur de commandes: interface entre l'utilisateur et le système d'exploitation
- ✓ Offre un langage de commande et est caractérisé par un environnement
- ✓ Accessible à travers :
 - Les consoles virtuelles
 - L'application « Terminal » en mode graphique
- ✓ Plusieurs Shells ont été implémentés: Bourne Shell (sh), le Korn Shell (ksh), C-Shell (csh), Bourne Again Shell (Bash),...
- ✓ Gestion d'un historique
- ✓ Personnalisation de l'environnement utilisateur
- ✓ Langage de programmation
- ✓ Variables d'environnement : HOME, PWD, ...
- ✓ Il existe 2 types de commandes :
 - ✓ **Les commandes internes** : sous-programmes de l'interpréteur de commandes (shell). Elles sont directement exécutables sans création d'un shell-fils. Exemples : alias, set, ...
 - ✓ **Les commandes externes** : fichiers exécutables. Leur exécution nécessite la création d'un processus fils. Exemples : mkdir, mv, chmod, ...

Caractéristiques (5)

- ✓ Format d'une ligne de commande :

<prompt> commande [options] arguments

- ✓ <prompt> : invite de commande pouvant avoir plusieurs formats tel que « [nom d'utilisateur @ répertoire courant] \$ »

- ✓ Exemples

Pour récupérer l'identifiant de l'utilisateur courant :

\$whoami → donne le login

\$id → donne toutes les informations

\$who → liste tous les utilisateurs connectés

Pour obtenir de la documentation sur une commande quelconque :

\$man <commande>

\$apropos <commande>

\$whatis <commande>

Pour afficher un message à l'écran :

\$echo hello

Caractéristiques (6)

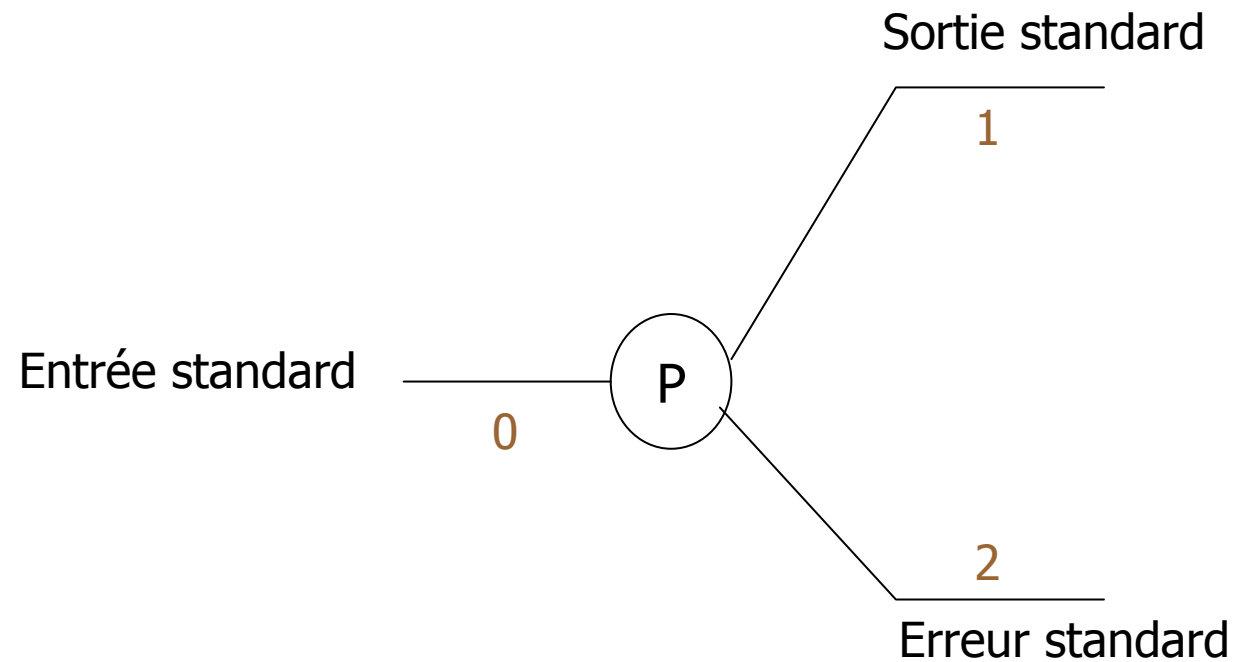
✓ Les processus

- ☞ Un processus est une instance d'un programme en exécution
- ☞ Chaque processus possède un environnement et un contexte
- ☞ Les processus s'exécutent aussi en arborescence: un processus peut créer un autre et être lui même « *fils* » d'un autre processus
- ☞ Un processus possède un identifiant (PID), un père (PPID), un propriétaire (UID) et un groupe propriétaire (GID)

☞ **ps**: liste les caractéristiques des processus

Caractéristiques (7)

- ✓ Les processus: les canaux standards



Caractéristiques (8)

✓ Les processus: les redirections

☞ Rediriger la sortie:

↪ Cde > fichier: crée fichier ou écrase son contenu pour y écrire le résultat de cde

↪ Cde >> fichier: crée fichier ou écrit à sa fin le résultat de cde.

☞ Rediriger l'entrée:

↪ Cde < fichier: cde prend ses arguments de fichier

☞ Rediriger l'erreur

↪ Cde 2> fichier ou Cde 2>> fichier

✓ Les processus : composition

☞ Composition séquentielle: C1;C2;C3

☞ Composition parallèle de processus indépendants C1&C2&C3

☞ Composition de processus communicants C1 | C2 | C3 : les pipes

Exp: who | sort

Installation d'un système Linux

- ✓ Système hébergé complètement en natif sur le disque
 - ✓ Installation complète sur tout le disque
 - ✓ Installation en « dual boot » : cohabitation entre Linux et un autre système d'exploitation (Windows ...)
- ✓ Installation en tant que **machine virtuelle**
 - ✓ Accéder à Linux à partir de Windows
 - ✓ Charger deux systèmes en même temps et profiter simultanément des deux environnements
 - ✓ Nécessite un logiciel de virtualisation exp : **VirtualBox** (logiciel libre) ou **VMware**
- ✓ Aucune installation → LiveCD
 - ✓ Certaines distributions offrent des versions bootables directement à partir d'un CD/DVD (Ubuntu, Fedora etc)
 - ✓ Ne requière pas d'installation sur disque
 - ✓ Tout l'environnement sera chargé à partir du CD

Pour finir

✓ Les distributions gratuites phares (documentation et téléchargement)

- ☞ **Ubuntu** : <http://ubuntu-fr.org>
- ☞ **Fedora** : <http://www.fedora-fr.org>
- ☞ **OpenSuse** : <http://fr.opensuse.org>
- ☞ **Mandriva** : <http://www2.mandriva.com>
- ☞ **CentOS** : <http://www.centos.org>

✓ Quelques commandes à tester :

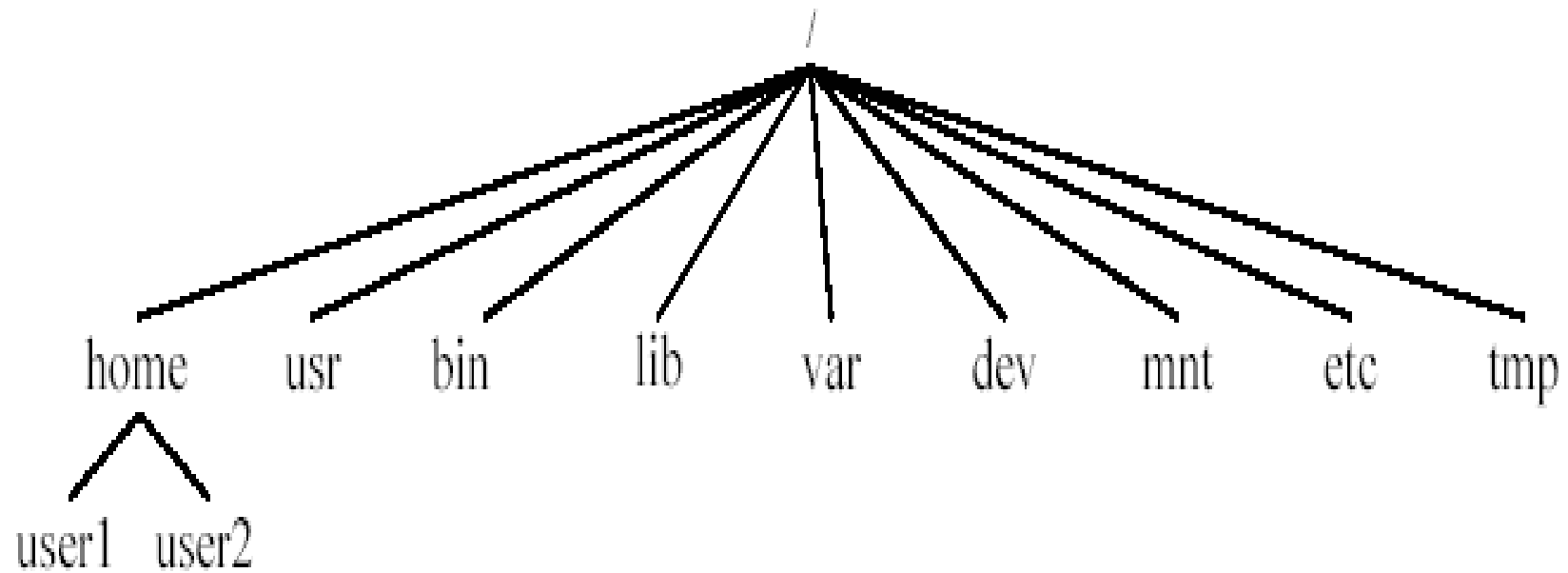
- ☞ man
- ☞ id
- ☞ who
- ☞ logname
- ☞ groups
- ☞ passwd
- ☞ clear
- ☞ history
- ☞ ...

2. Le système de gestion de fichiers

Présentation générale

- ✓ Dans Unix la notion de fichier ne se limite pas à la simple notion usuelle de fichier disque
- ✓ Un fichier apparaît comme un objet typé et pourra correspondre à :
 - ☞ Un fichier disque au sens classique
 - ☞ Une ressource (physique ou logique) du système (*device*)
 - ☞ Exemple: terminaux, imprimantes, disques physiques ou logiques, la mémoire physique ou la *kernel memory* (contient les tables du noyau)
- ✓ SGF : Principe similaire aux autres systèmes: arborescence de répertoires et de fichiers
 - ☞ Plusieurs implémentations et types de systèmes de fichiers: Ext2, Ext3, GFS, ReiserFs,...

Arborescence (1)



Arborescence (2)

✓ Répertoires standards

- 👉 / : répertoire racine du système de fichiers
- 👉 /root : répertoire de travail par défaut de l'utilisateur **root**
- 👉 /tmp: répertoire des fichiers temporaires
- 👉 /bin: répertoire des programmes accessibles à tous les users
- 👉 /sbin: répertoire des programmes accessibles au root uniquement
- 👉 /etc: contient des fichiers de configuration de services
- 👉 /lib: bibliothèques partagées et modules nécessaires au chargement du système
- 👉 /var: fichiers d'historiques (log), mails, ...
- 👉 /home: les répertoires de travail des utilisateurs
- 👉 /boot: le noyau du système et fichiers du démarrage
- 👉 /dev: fichiers des périphériques
- 👉 /usr , /proc,

Chemin relatif et chemin absolu

- ✓ Le chemin absolu: noter un fichier ou un répertoire par le chemin complet à partir de la racine

👉 Exemple: **/home/user1/test**

- ✓ Le chemin relatif: noter un fichier ou un répertoire par rapport à la position courante

Rép. Courant = /home => user1/test

pwd: retourne le répertoire en cours

- ✓ **..** : note le répertoire parent
- ✓ **.** : note le répertoire courant
- ✓ **~** : note le répertoire de travail de l'utilisateur en cours

- ✓ Exemples:

👉 **pwd = /home/user1**

👉 **cd /home/user2 ⇔ cd ../user2**

Types de fichiers

✓ **File**: retourne le type du fichier

- ☞ Fichiers ordinaires (-)
- ☞ Répertoires (d)
- ☞ Liens symboliques (l)
- ☞ Fichiers block (b)
- ☞ Fichiers caractères (c)

✓ Exemple:

```
$ ls -l /dev  
brw-rw---- 1 root disk 3, 2 Jul 5 2000 hda :périphérique block  
crw-rw-rw- 1 root tty 4, 9 Jul 5 2000 tty9 :périph. caractère  
lrwxrwxrwx 1 root disk 3, 2 Jul 5 2000 cdrom -> hda : lien
```

Manipulation des répertoires

- ✓ Les noms des répertoire et des fichiers sont sensibles à la casse
- ✓ Un nom de répertoire ou de fichier commençant par '.' est un fichier ou un répertoire caché
- ✓ Création: **mkdir** nom_rep1 [nom_rep2 ... nom_repN]
- ✓ Destruction :
 - ☞ **rmdir**: efface un répertoire vide
 - ☞ **rm -r**: efface récursivement le contenu d'un répertoire puis celui-ci
- ✓ Listage: **ls**
 - ☞ **ls -a**: lister aussi les fichiers et les répertoires cachés
 - ☞ **ls -l**: lister le contenu avec affichage de plus d'informations

```
$ ls -l =>
```

```
drwxr-xr-x 1 user1 users 112 oct 12 09:07 premier
```

```
=> d : indique un répertoire; les permissions; nombre de liens; utili.  
propriétaire; groupe propriétaire; taille; date de dernière modif., nom
```

Manipulation des fichiers (1)

- ✓ Création:

- 👉 **touch nom_fichier**: crée un fichier vide

- 👉 Utiliser un éditeur de texte: **vi, pico, emacs...**

- ✓ Suppression: **rm**

- ✓ Copie: **cp fichier_source fichier_dest**

- ✓ Déplacement: **mv fichier destination**

- ✓ Renommage: **mv fichier nouv_nom**

- ✓ Affichage du contenu d'un fichier:

- 👉 **cat**

- 👉 **less**

- 👉 **more**

Manipulation des fichiers (2)

- ✓ Afficher n premières lignes d'un fichier:

head -n fichier

- ✓ Afficher n dernières lignes d'un fichier:

tail -n fichier

tail -f fichier : ouverture en fin de fichier avec visualisation en temps réel des modifications.

☞ Remarque: par défaut n=10

- ✓ Type de fichier: **file fichier**

- ✓ Juxtaposer les lignes des 2 fichiers:

paste [-d *carac. séparateur*] fichier1 fichier2 (en colonnes)

cat fichier1 >> fichier2 (les lignes de fichier1 après les lignes de fichier2)

Caractères spéciaux (1)

- ✓ * : remplace n (0 et +) caractères
- ✓ ?: remplace exactement 1 caractère

```
ls test* => affichage de: test, test1, test10, test254, ...  
ls test? => affichage de: test1
```

- ✓ \$: interprété comme la valeur de la variable qui vient après

```
echo HOME => affiche le mot HOME  
echo $HOME => affiche : /home/user1
```

Caractères spéciaux (2)

- ✓ ` ` : tous ce qui sera mis entre ces cotes sera considéré comme étant une commande et sera exécuté
- ✓ “ ” : tous ce qui trouve entre ces doubles cotes sera considéré comme une chaîne de caractères sauf les caractères spéciaux
- ✓ ‘ ’ : tous ce qui trouve entre ces cotes sera considéré comme une chaîne de caractères
- ✓ \ : annule l'effet du caractère spécial qui vient juste après

Caractères spéciaux (3)

✓ Exemples

echo "mon répertoire est \$PWD" =>

affiche: mon répertoire est /home/user1

echo 'mon répertoire est \$PWD' =>

affiche: mon répertoire est \$PWD

echo `date` => 04:09:2006

echo le prix est 10 \\$ => affiche : le prix est 10\$ (le caractère \ protège \$)

Les inodes (1)

Inode= descripteur d'un fichier dans l'arborescence

Structure composée de plusieurs champs d'informations et de plusieurs pointeurs.

Informations :

Inode = structure {

Num numéro d'inode unique dans l'arborescence

Nom le nom du fichier

Mode les permissions du fichier

UID l'utilisateur propriétaire de ce fichier

GID le groupe qui possède ce fichier

Taille la taille du fichier en octets

ATIME date de dernier accès

MTIME date de dernière modification du fichier

CTIME date des dernières modifications de l'inode

...}

Les inodes (2)

- ✓ Chaque inode contient aussi un ou plusieurs pointeurs vers un ou plusieurs structures fichiers.
- ✓ Chaque inode possède un numéro unique
- ✓ Pour visualiser les informations contenues dans l'inode associée à un fichier

stat nom_fichier

- ✓ **ls -i** permet de lister les fichiers avec leur numéros d'inodes correspondant

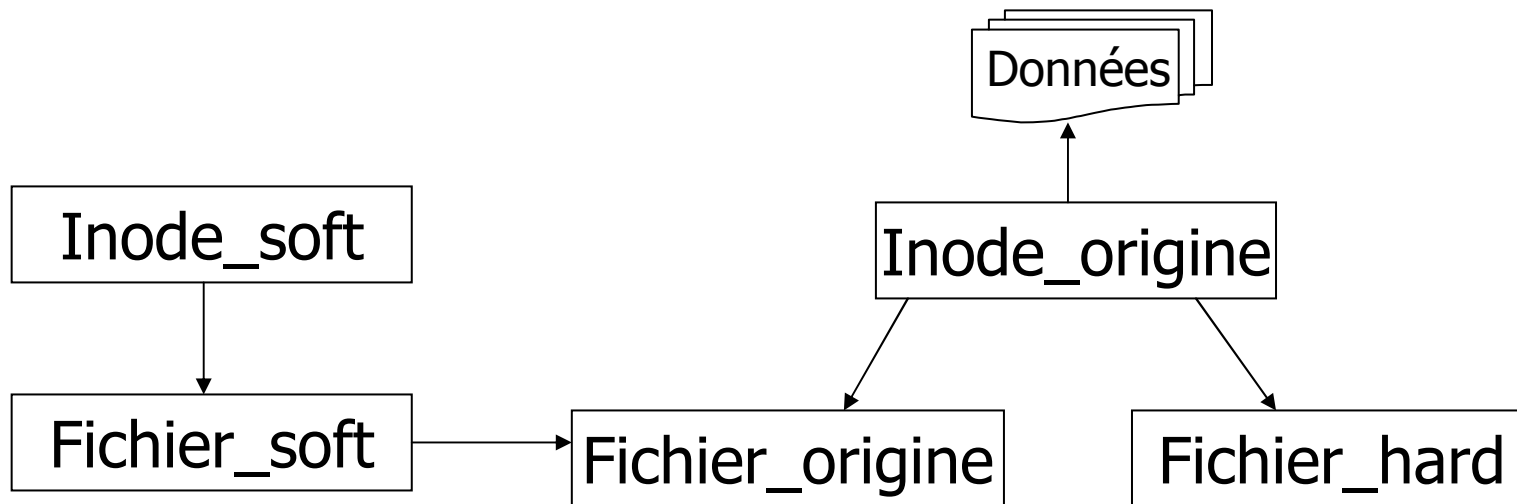
Les liens (1)

- ✓ **Lien symbolique** (soft): un nouveau fichier ayant une nouvelle inode qui pointe vers le fichier original (contient son chemin d'accès).
- ✓ **Lien physique** (hard): un nouveau pointeur de l'inode du fichier original vers une nouvelle structure fichier ⇒ une manière de donner plusieurs noms à un fichier sans dupliquer ses données disque et sans création d'une nouvelle inode.

```
$ ln fichier lien_hard  
$ ln -s fichier lien_soft
```

- ✓ Fichier original et lien physique identiques
- ✓ Une inode persiste jusqu'à destruction du dernier lien physique vers lequel elle pointe

Les liens (2)



- ✓ Si on supprime le fichier original le lien symbolique pointe vers le vide
- ✓ Cependant, le lien physique pointe toujours vers les données originales et garde sa consistance

Autres commandes sur les fichiers (1)

✓ **sort**

☞ trie l'entrée ligne par ligne

☞ options: -r inverse l'ordre de tri

 +n : ignore les n 1ers champs

 -n : tri numérique

 -u : afficher une instance unique de chaque ligne

 -b : ignorer les blancs en début des champs

 -f : confond les lettres majuscules et minuscules

Autres commandes sur les fichiers (2)

✓ **uniq**

☞ Affiche un fichier en détruisant les lignes **consécutives** dupliquées

☞ options: -u (affiche les lignes "uniques"),
 -d (affiche les lignes "dupliquées")

✓ **wc**

☞ compter le nombre de lignes, le nombre de mots et le nombre de caractères d'un fichier.

☞ **wc -l fichier** : retourne le nombre de lignes de fichier

☞ **wc -w fichier** : retourne le nombre de mots de fichier

☞ **wc -c fichier** : retourne le nombre de caractères de fichier

Autres commandes

✓ **paste : permet de concaténer deux fichiers en colonne**

✓ Exemple :

paste -d% test1 test2 : génère un texte construit par le contenu de test1 dans la 1^{ère} colonne et test2 en deuxième colonne avec le séparateur de colonnes « % »

✓ Compression de fichiers : gzip, bzip, compress, ..

✓ Décompression de fichiers : gunzip, bunzip, uncompress, ..

✓ Archivage : tar

✓ Montage / démontage de partitions ou de volumes disques : mount /
umount

✓ ...

3. Sécurité du système de fichiers

Introduction

- ✓ Chaque élément du SGF appartient à un utilisateur, est associé à un groupe et porte des permissions définissant les droits d'accès pour chaque type d'utilisateur.
- ✓ Par défaut (à la création), l'utilisateur propriétaire d'un élément sera celui qui l'a créé. Le groupe propriétaire est le groupe par défaut de l'utilisateur propriétaire.
- ✓ L'utilisateur propriétaire a le droit de modifier les droits d'accès de ses fichiers et ses répertoires.
- ✓ L'utilisateur propriétaire a le droit de modifier l'appartenance d'un de ses éléments --> il le donne à un autre utilisateur et perd par conséquent ses droits.
- ✓ Exception : **root** a tous les droits sur tous les fichiers même ceux d'autres utilisateurs.

La commande `ls -l` donne ces informations :

```
$ ls -l premier
```

```
-rwxr-xr-x 1 user1 users 112 oct 12 09:07 premier
```


Les permissions standards (1)

- ✓ Pour chaque élément du SGF, on distingue 3 types d'utilisateurs, chaque type aura des permissions ou autorisations spécifiques.
- ✓ 3 permissions standards sont définies, chaque type d'utilisateur aura une combinaison parmi ces trois.
- ✓ Les permissions peuvent être représentées par des lettres ou numériquement en sommant les chiffres associées aux permissions de chaque type d'utilisateur.

3 types d'utilisateurs:

- ✓ u: utilisateur Propriétaire
- ✓ g: groupe propriétaire
- ✓ o: (others) : autres

3 types d'autorisations:

- ✓ r (read)= 4
- ✓ w (write) =2
- ✓ x (execute) =1

Les permissions standards (2)

✓ Exemple :

```
$ ls -l premier
```

```
-rwxr-xr-x 1 user1 users 112 oct 12 09:07 premier
```

✓ Le champs permissions : **rwxr-xr-x**

- ✓ 3 premiers caractères: permissions pour le propriétaire (user1) : **rwx** (lire, écrire et exécuter) --> somme numérique =7
- ✓ 3 caractères suivants: permissions pour le groupe propriétaire (users) : **r-x** => lire et exécuter mais pas de modification (w non accordée) --> somme numérique =5
- ✓ 3 derniers caractères : pour les autres utilisateurs : **r-x** --> somme numérique =5

✓ Équivalent numérique des permissions standards de ce fichier : **rwxr-xr-x** = 755

Les permissions standards (3)

- ✓ Signification des permissions :

Permission	Pour les fichiers	Pour les répertoires
r	Droit de lecture (cat, less, etc)	Droit de lister le contenu (ls)
w	Droit d'écriture (modification du contenu par un éditeur)	Droit de créer de nouveaux éléments à l'intérieur (mkdir, touch, etc)
x	Droit d'exécution (nécessaire pour les binaires et les scripts)	Droit d'accès (se positionner à l'intérieur par cd)

Modification des permissions

- ✓ Chaque utilisateur a le droit de modifier les permissions de ses fichiers et de ses répertoires (**root** peut tout changer)

- ✓ Changer les permissions : **chmod**

\$ chmod num fichier

☞ num: équivalent numérique des nouvelles permissions qu'on veut appliquer sur fichier

☞ Exemple: **chmod 644 fichier** $\Leftrightarrow$ **rw-r--r--**

\$ chmod [u | o | g] $\pm$ r | w | x fichier

☞ ajoute ou élimine une permission pour un type d'utilisateur donné

☞ Exemples :

☞ **\$ chmod g+w fichier** : ajoute la permission d'écrire pour les membres du groupe propriétaire.

☞ **\$ chmod o-r fichier** : supprime la permission de lire pour les «autres».

☞ **\$ chmod +x fichier** : rend le fichier exécutable par tout le monde (u+g+o)

Gestion d'appartenance

- ✓ Chaque utilisateur peut modifier l'appartenance de ses éléments (root peut changer l'appartenance de n'importe quel élément)
- ✓ Changer l'utilisateur propriétaire : **chown**
 - ☞ Changer l'utilisateur propriétaire uniquement
\$ **chown user2 fichier**
 - ☞ Changer le propriétaire et le groupe propriétaire :
\$ **chown user2.users fichier**
- ✓ Changer le groupe propriétaire : **chgrp**
 - ☞ \$ **chgrp users fichier**

Gestion des utilisateurs

- ✓ 3 types d'utilisateurs :
 - ✓ L'administrateur du système : **root**
 - ✓ Les utilisateurs système
 - ✓ Les utilisateurs non privilégiés
- ✓ Le fichier **/etc/passwd** contient la liste des utilisateurs déclarés dans le système
- ✓ **root** peut créer de nouveaux utilisateurs non privilégiés.
 - ✓ **\$useradd** nom_utilisateur → créer un nouveau utilisateur
 - ✓ **\$passwd** nom_utilisateur → donner ou changer le mot de passe d'un utilisateur
 - ✓ **\$passwd** → changer le mot de passe de l'utilisateur courant
- ✓ Identité de l'utilisateur de la session courante :
 - ✓ **\$id**
 - ✓ **\$whoami**
- ✓ **\$userdel** nom_utilisateur → supprime l'utilisateur du système ainsi que son mot de passe sans supprimer son répertoire de travail

Gestion des groupes

- ✓ Un utilisateur peut appartenir à plusieurs groupes :
 - ✓ Le groupe initial ou par défaut : porte le même nom de l'utilisateur
 - ✓ Possibilité de création d'autres groupes et d'ajout d'utilisateurs
- ✓ Le fichier **/etc/group** contient la liste des groupes déclarés dans le système
- ✓ **root** peut créer de nouveaux groupes.
 - ✓ **\$groupadd nom_groupe** → créer un nouveau groupe
 - ✓ **\$usermod -G nom_groupe nom_utilisateur** → ajoute l'utilisateur à un groupe
- ✓ Supprimer un groupe (sans supprimer les utilisateurs) : **\$ groupdel nom-groupe**

Les permissions spéciales

✓ Permissions spéciales:

- 👉 **Setuid:** Applicable sur un fichier exécutable. L'utilisateur exécutant le fichier aura les mêmes privilèges que l'utilisateur propriétaire.

\$ chmod u+s fichier

\$ ls -l fichier

-rwsr-xr-x (.....) fichier

- 👉 **Setgid:** Applicable sur un répertoire. Tout élément créé à l'intérieure appartiendra au groupe propriétaire du répertoire.

\$ chmod g+s rep

\$ ls -l

drwxr-sr-x (.....) rep

- 👉 **Sticky bit:** Applicable sur un répertoire. Les éléments à l'intérieure peuvent être accessibles en lecture et écriture mais ne peuvent être supprimés que par leurs propriétaires.

\$ chmod +t rep

\$ ls -l

drwxr-xr-t (.....) rep

Les permissions par défaut

- ✓ Les fichiers et les répertoires nouvellement créés portent des **permissions standards** par défaut
- ✓ Généralement, même valeur pour tous les utilisateurs non privilégiés et une valeur spéciale pour **root**.
- ✓ Le complémentaire des permissions standards par défaut est stocké dans une variable système appelée UMASK
- ✓ UMASK : valeur sur 4 chiffres
 - ☞ Le 1er : 0 indique la forme octale des permissions initiales
 - ☞ Les 3 derniers : complément de 666 des permissions initiales
 - ☞ Pour les répertoires, la permission **x** est ajoutée par défaut à la création
- ✓ Afficher le masque pour l'utilisateur en cours:
 - ☞ **umask** : permissions sous forme octales
 - ☞ **umask -S** : sous forme symbolique (alphabétique)
- ✓ Changer le masque (des permissions standards) : **umask nnn**
 - ☞ **\$ umask 022**: les fichiers créés auront par défaut les permissions 644 et les répertoires 755
 - ☞ **\$ umask 000**: tous les fichiers qui seront créés auront les permissions 666, les répertoires 777